Best Practices of Software Engineering

R

Copyright © 1998 Rational Software, all rights reserved

Objectives: Best Practices of Software Engineering

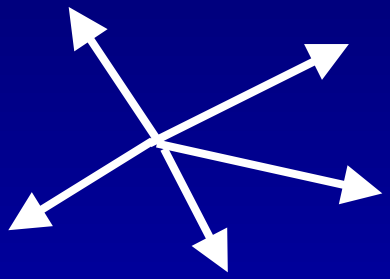
- Symptoms and root causes of software development problems
- Best practices for software development
- Look at how best practices address the root causes of software development problems

Copyright © 1998 Rational Software, all rights reserved

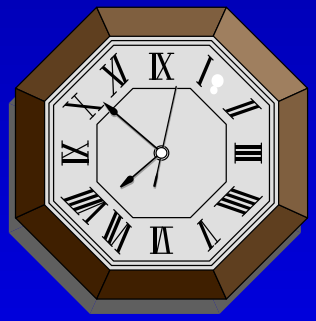
R

Software Development Situation Analysis

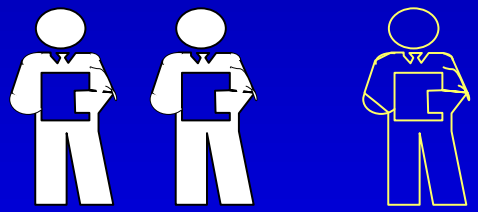
World increasingly software dependent



Applications expanding in size, complexity, & distribution



Demands for increased productivity & quality in less time



Lack of qualified people

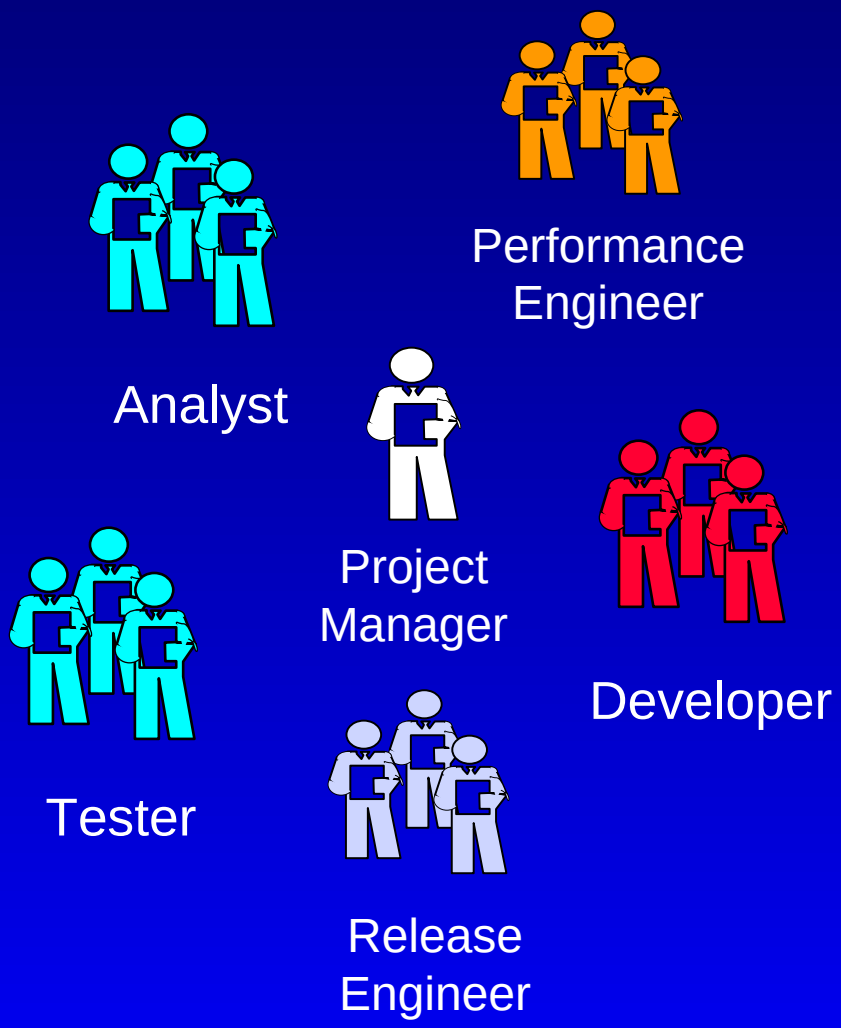
Copyright © 1998 Rational Software, all rights reserved

R

Software Development is a Job for Teams

Challenges

- Larger teams
- Specialization
- Distribution
- Rapid technology change



Copyright © 1998 Rational Software, all rights reserved

Symptoms of Software Development Problems

- Poor understanding of end-user needs
- Difficulty with changing requirements
- Modules don't fit together
- Software hard to maintain or extend
- Late discovery of serious project flaws
- Poor software quality
- Unacceptable software performance
- Team members in each other's way
  - Unable to reconstruct who changed what, when, where, why
- Untrustworthy build-and-release process

Copyright © 1998 Rational Software, all rights reserved

R

Treating Symptoms Not Effective

Symptoms

- end-user needs
- changing requirements
- modules don't fit
- hard to maintain
- late discovery
- poor quality
- poor performance
- colliding developers
- build-and-release

Diagnose

Root Causes

- insufficient requirements
- ambiguous communications
- brittle architectures
- overwhelming complexity
- undetected inconsistencies
- poor testing
- subjective assessment
- waterfall development
- uncontrolled change
- insufficient automation

Copyright © 1998 Rational Software, all rights reserved

R

Root Causes of Software Development Problems

- Insufficient requirements management
- Ambiguous and imprecise communications
- Brittle architectures
- Overwhelming complexity
- Undetected inconsistencies among requirements, designs, and implementations
- Insufficient testing
- Subjective project status assessment
- Delayed risk reduction due to waterfall development
- Uncontrolled change propagation
- Insufficient automation

Copyright © 1998 Rational Software, all rights reserved

R

Best Practices Address Root Causes

Root Causes

- ☑ Insufficient requirements
- ☑ Ambiguous communications
- ☑ Brittle architectures
- ☑ Overwhelming complexity
- ☑ Subjective assessment
- ☑ Undetected inconsistencies
- ☑ Poor testing
- ☑ Waterfall development
- ☑ Uncontrolled change
- ☑ Insufficient automation

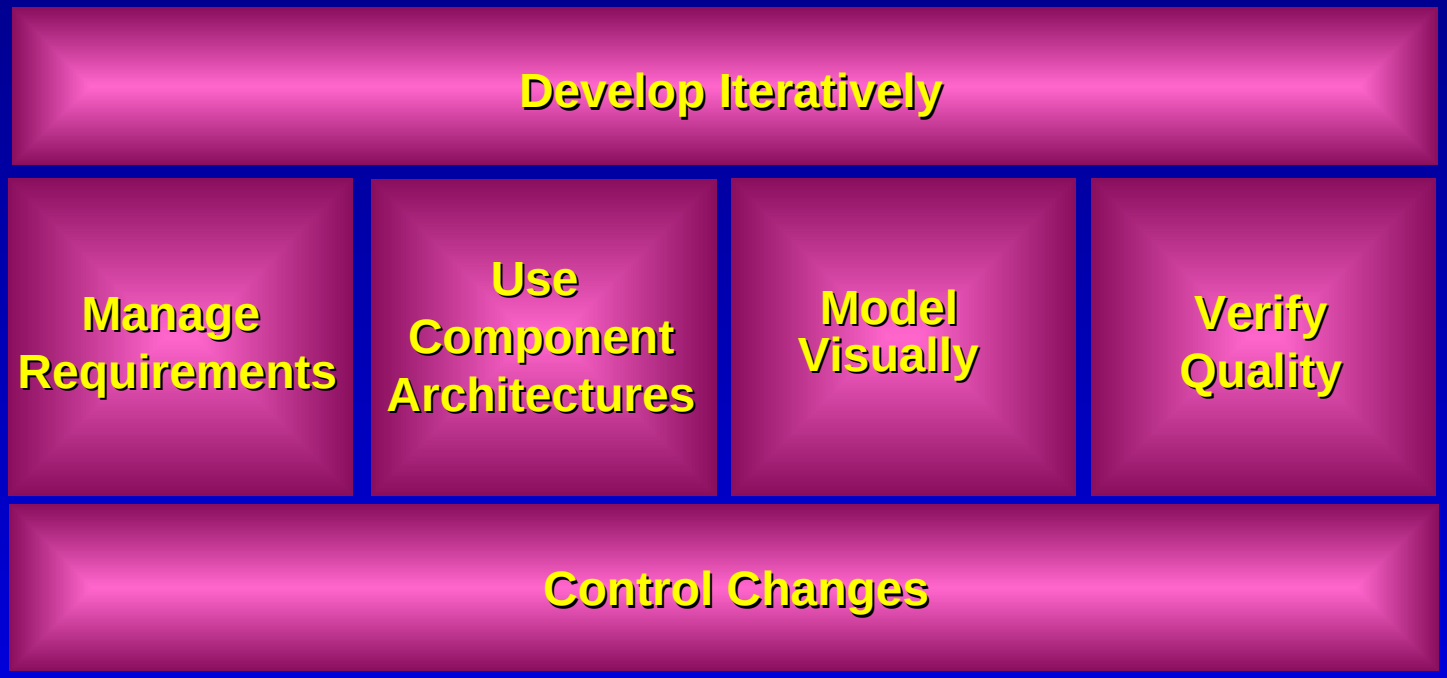
Best Practices

- ☑ Develop iteratively
- ☑ Manage requirements
- ☑ Use component architectures
- ☑ Model the software visually
- ☑ Verify quality
- ☑ Control changes

Copyright © 1998 Rational Software, all rights reserved

R

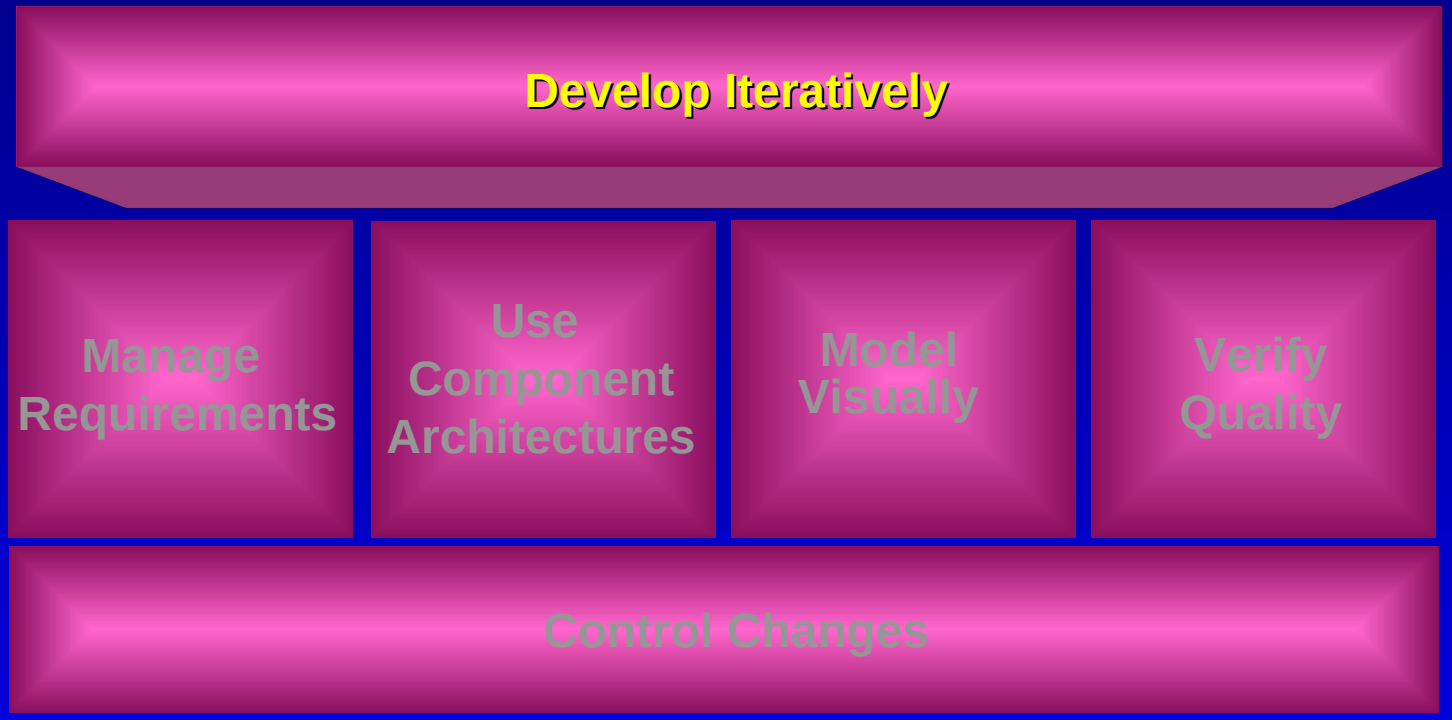
Best Practices of Software Engineering



Copyright © 1998 Rational Software, all rights reserved

R

Practice 1: Develop Software Iteratively

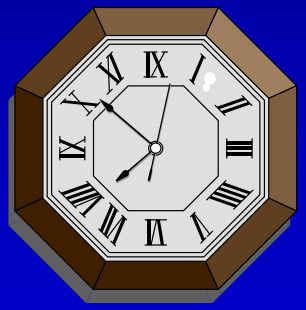


Copyright © 1998 Rational Software, all rights reserved

R

Practice 1: Develop Software Iteratively

- An initial design will likely be flawed with respect to its key requirements
- Late-phase discovery of design defects results in costly over-runs and/or project cancellation

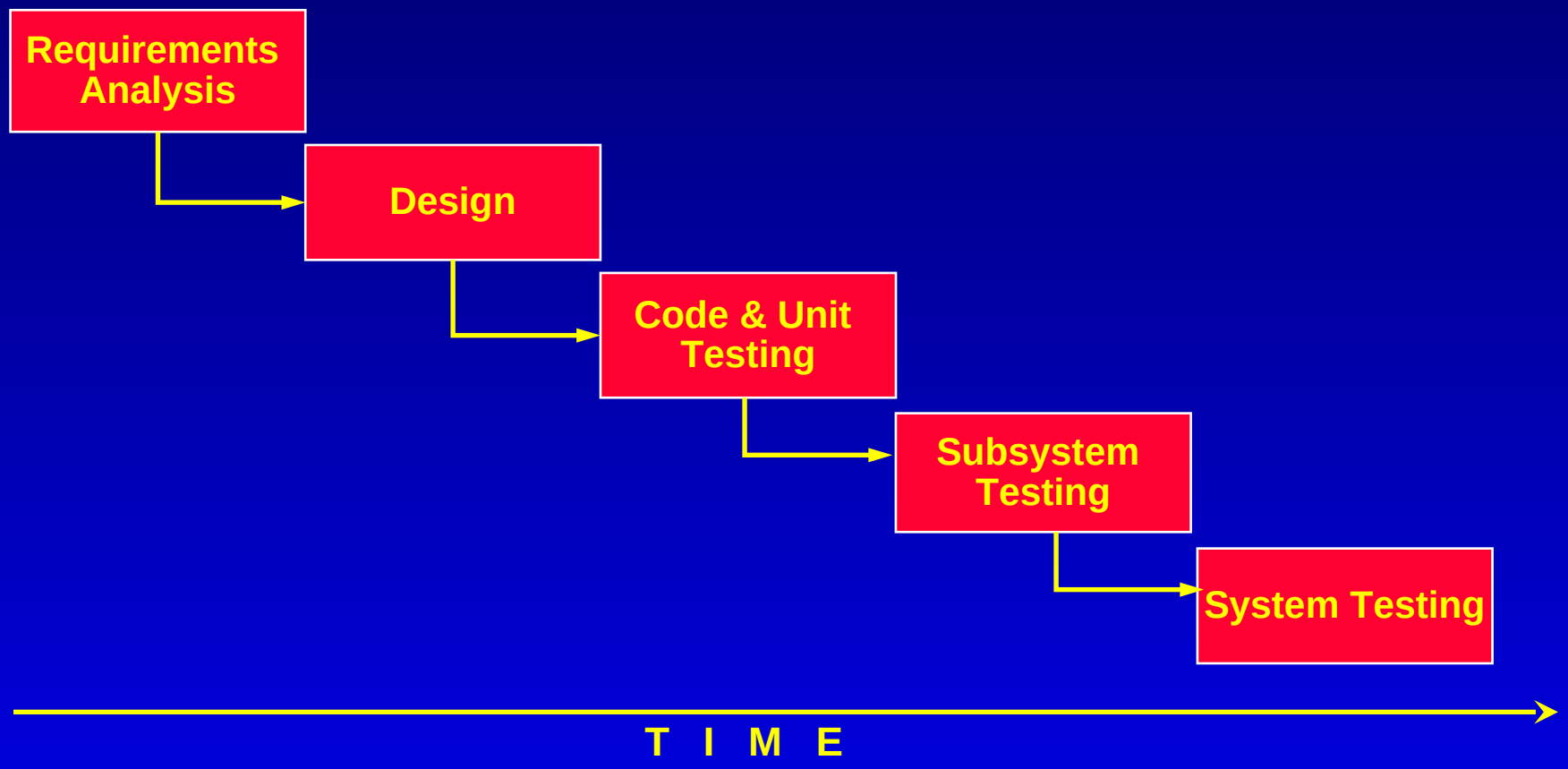


The time and money spent implementing a faulty design are not recoverable

Copyright © 1998 Rational Software, all rights reserved

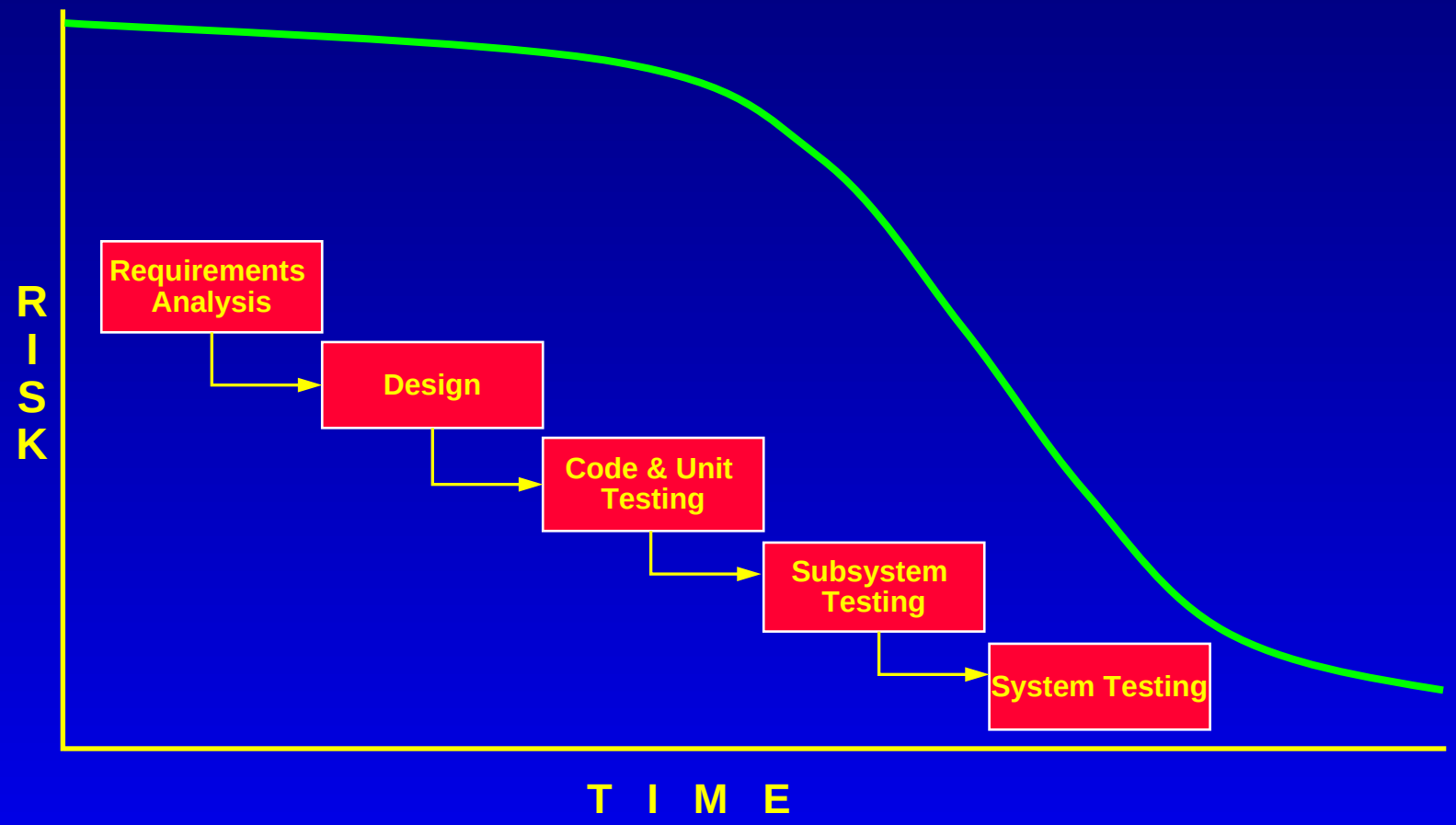
R

Traditional Waterfall Development



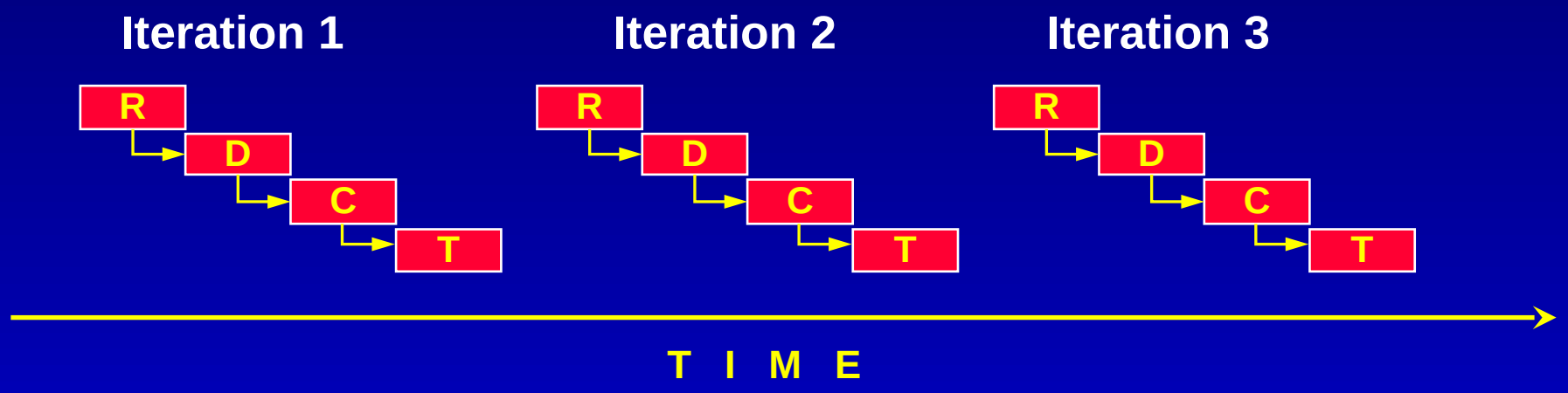
Copyright © 1998 Rational Software, all rights reserved

Waterfall Development Delays Reduction of Risk



Copyright © 1998 Rational Software, all rights reserved

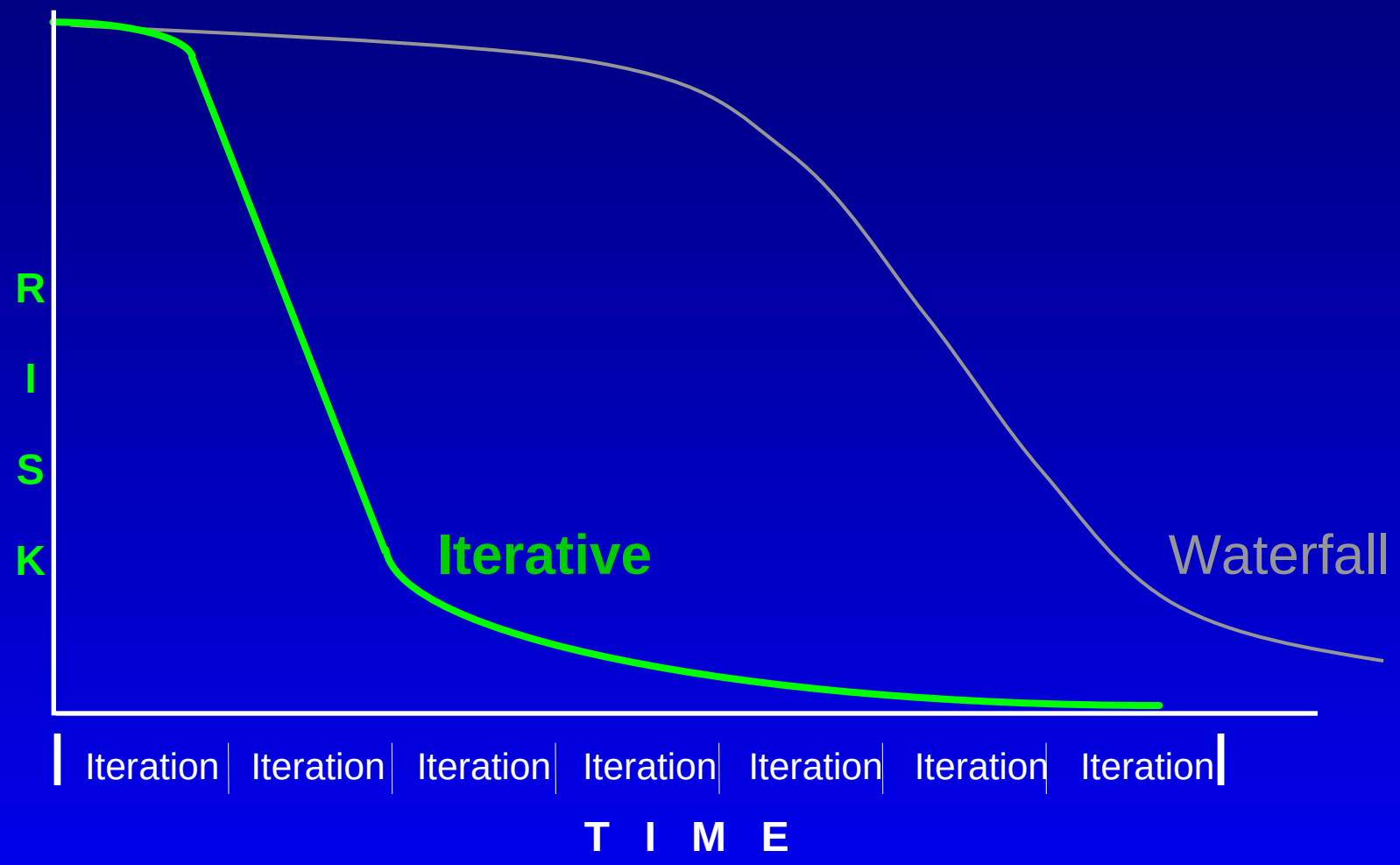
Apply the Waterfall Iteratively to System Increments



- Earliest iterations address greatest risks
- Each iteration produces an executable release, an additional increment of the system
- Each iteration includes integration and test

Copyright © 1998 Rational Software, all rights reserved

Iterative Development Accelerates Risk Reduction



Copyright © 1998 Rational Software, all rights reserved

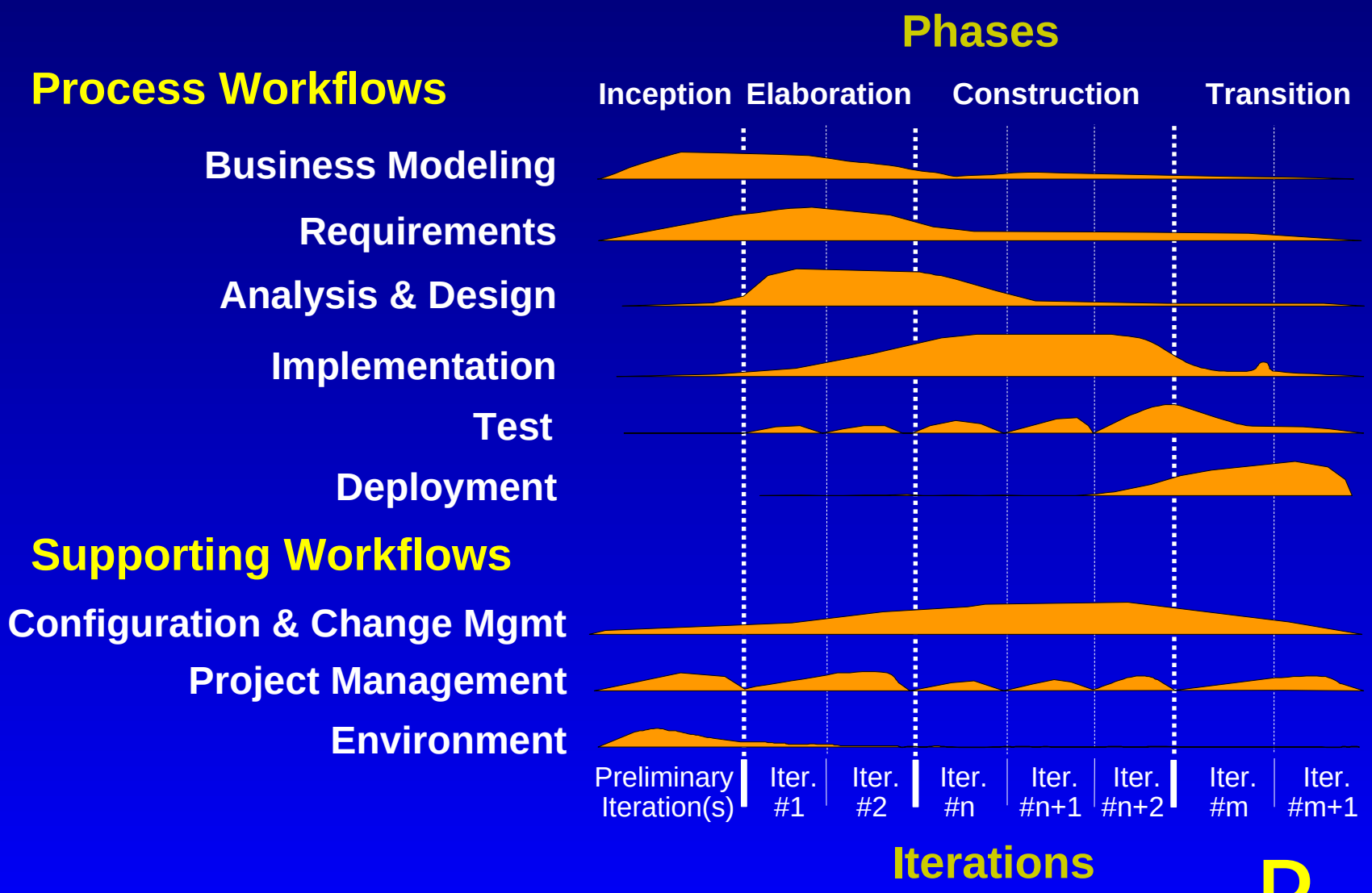
Iterative Development Characteristics

- Critical risks are resolved before making large investments
- Initial iterations enable early user feedback
- Testing and integration are continuous
- Objective milestones provide short-term focus
- Progress is measured by assessing implementations
- Partial implementations can be deployed

Copyright © 1998 Rational Software, all rights reserved

R

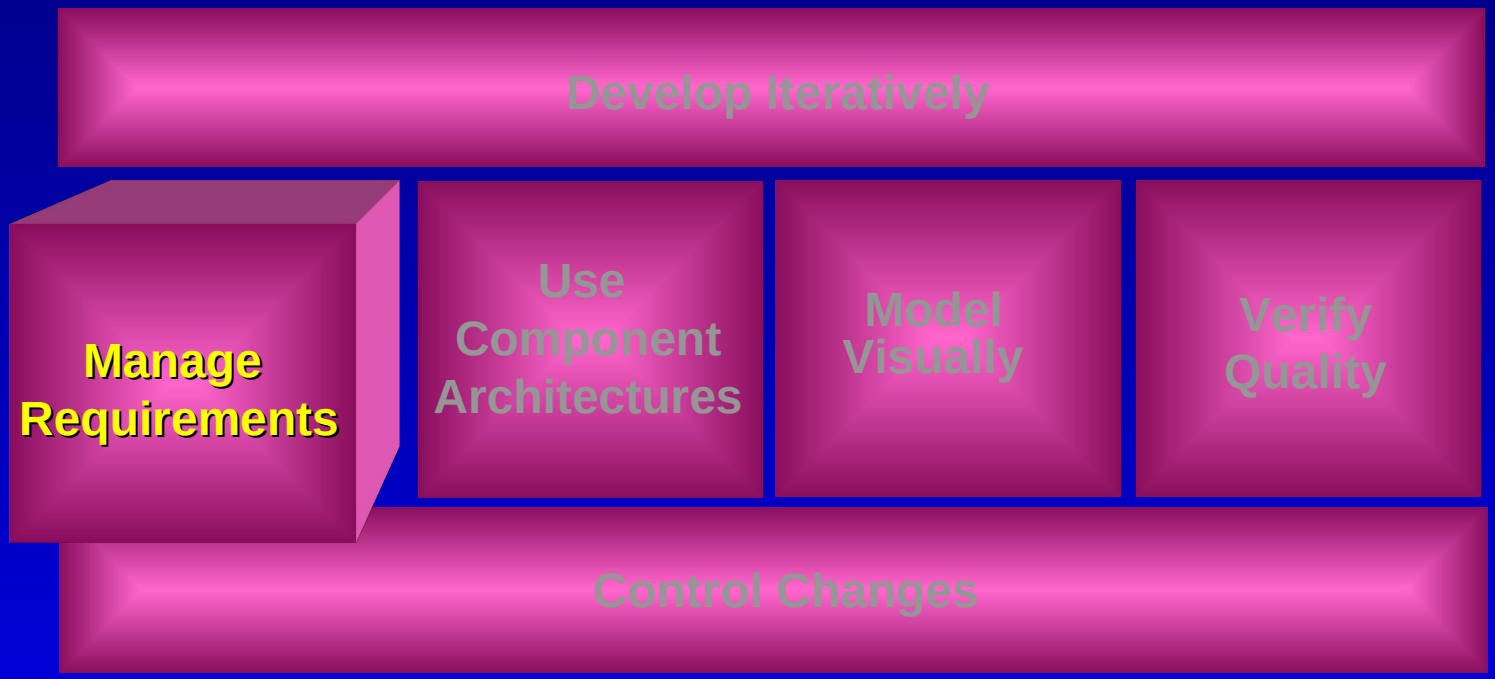
Apply Best Practices Throughout the Life Cycle



Unified Software Practices v 5.0 - C
Copyright © 1998 Rational Software, all rights reserved

R

Practice 2: Manage Requirements

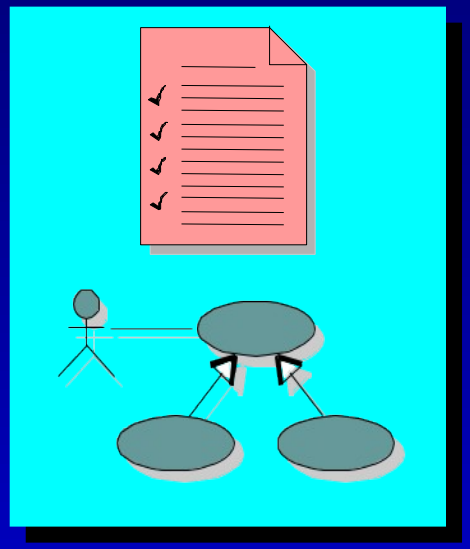


Copyright © 1998 Rational Software, all rights reserved

R

Practice 2: Manage Requirements

- Elicit, organize, and document required functionality and constraints
- Evaluate changes and determine their impact
- Track and document tradeoffs and decisions



Requirements are dynamic -- Expect them to change during software development

R

Copyright © 1998 Rational Software, all rights reserved

Definitions: Requirements and Their Management

- A **requirement** is a condition or capability to which the system must conform
- **Requirements management** is a systematic approach to
  - Eliciting, organizing, and documenting the requirements of the system, and
  - Establishing and maintaining agreement between the customer/user and the project team on the changing requirements of the system

Copyright © 1998 Rational Software, all rights reserved

R

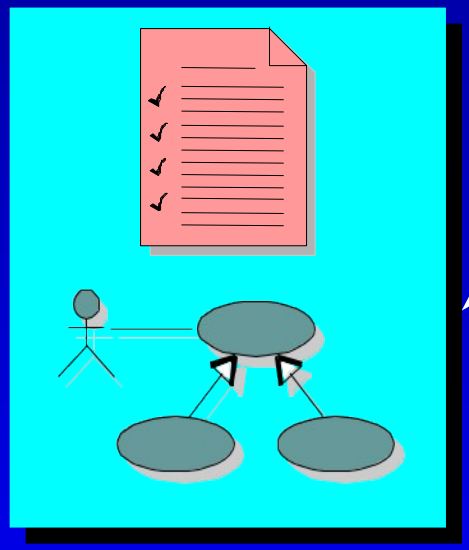
Agreement on What the System Should Do

Customer
User Community

The Goal

System
To Be Built

Requirements
Verification



Requirements

Surrogate
Goal

Adapted from Al Davis

Copyright © 1998 Rational Software, all rights reserved

R

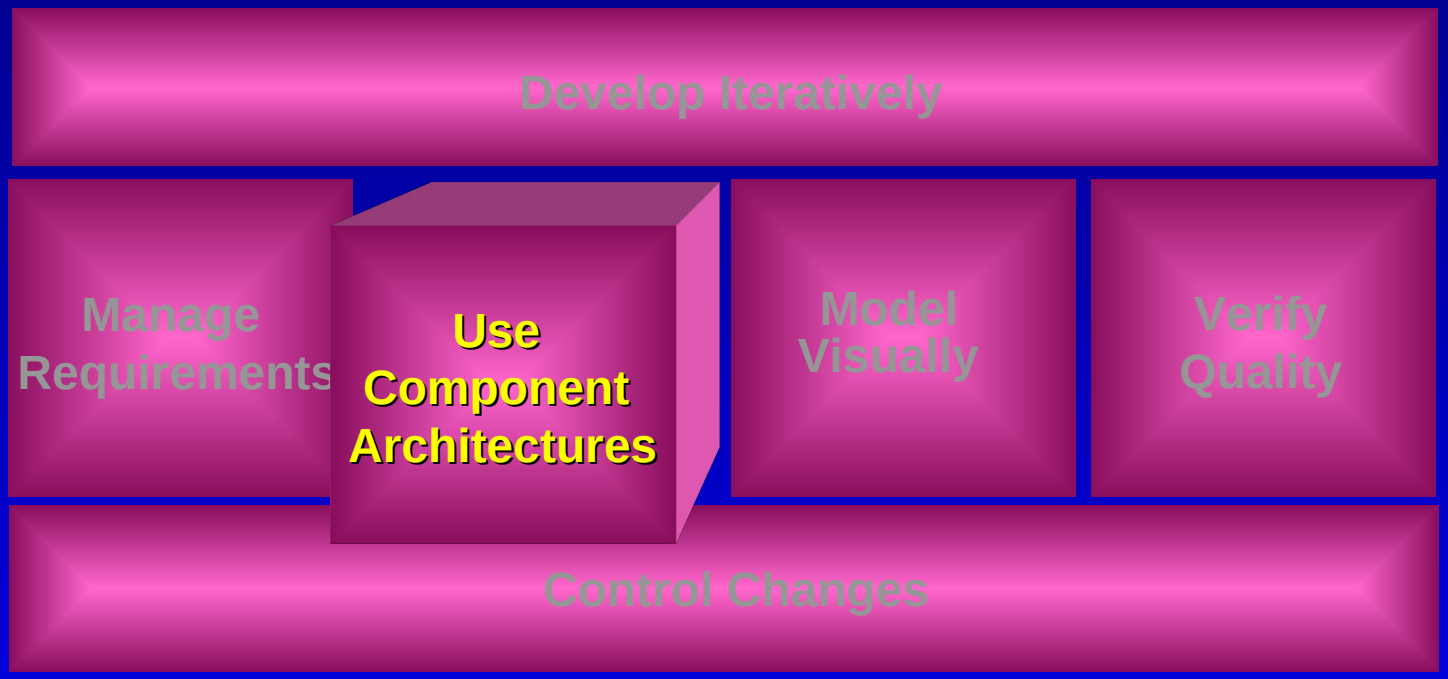
How To Catch Requirements Errors Early

- Effectively analyze the problem and elicit user needs
- Gain agreement with the customer/user on the requirements
- Model interaction between the user and the system
- Establish a baseline and change control process
- Maintain forward and backward traceability of requirements
- Use an iterative process

Copyright © 1998 Rational Software, all rights reserved

R

Practice 3: Use Component-Based Architectures



Copyright © 1998 Rational Software, all rights reserved

R

Software Architecture Defined

- **Software architecture** encompasses significant decisions about the organization of a software system
  - Selection of the structural elements and their interfaces by which a system is composed
  - Behavior as specified in collaborations among those elements
  - Composition of these structural and behavioral elements into progressively larger subsystems
  - Architectural style that guides this organization, these elements and their interfaces, their collaborations, and their composition

Copyright © 1998 Rational Software, all rights reserved

R

Architectural Concerns

- Software architecture is concerned with structure and behavior and context:
  - Usage
  - Functionality
  - Performance
  - Resilience
  - Reuse
  - Comprehensibility
  - Economic and technological constraints and tradeoffs
  - Aesthetics

Copyright © 1998 Rational Software, all rights reserved

R

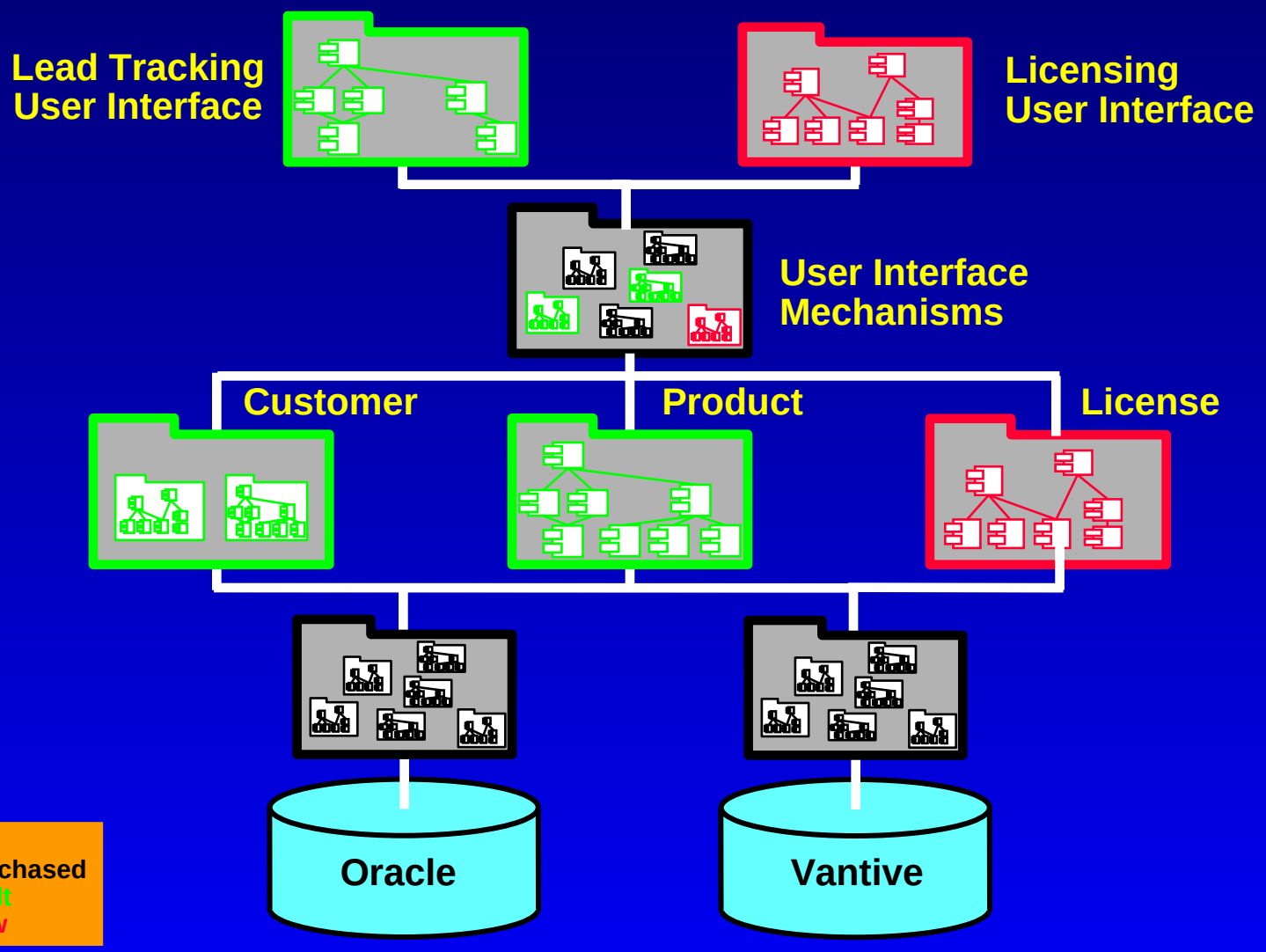
Resilient, Component-Based Architectures

- Good architectures meet their requirements, are **resilient,** and are **component-based**
- A **resilient** architecture enables
  - Improved maintainability and extensibility
  - Economically-significant reuse
  - Clean division of work among teams of developers
  - Encapsulation of hardware and system dependencies
- A **component-based** architecture permits
  - Reuse or customization of existing components
  - Choice of thousands of commercially-available components
  - Incremental evolution of existing software

R

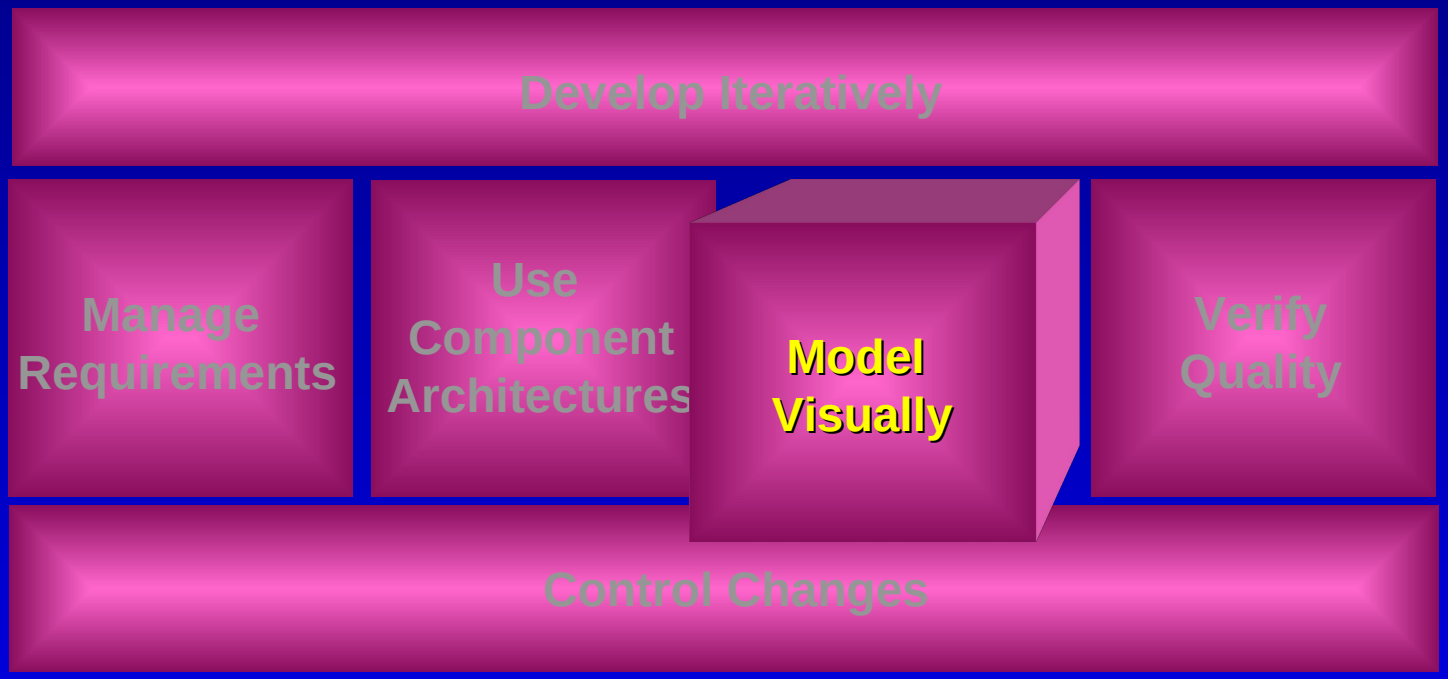
Copyright © 1998 Rational Software, all rights reserved

Example: Component-Based Architecture



Copyright © 1998 Rational Software, all rights reserved

Practice 4: Visually Model Software



Copyright © 1998 Rational Software, all rights reserved

R

Practice 4: Visually Model Software

- Capture the structure and behavior of architectures and components
- Show how the elements of the system fit together
- Hide or expose details as appropriate for the task
- Maintain consistency between a design and its implementation
- Promote unambiguous communication

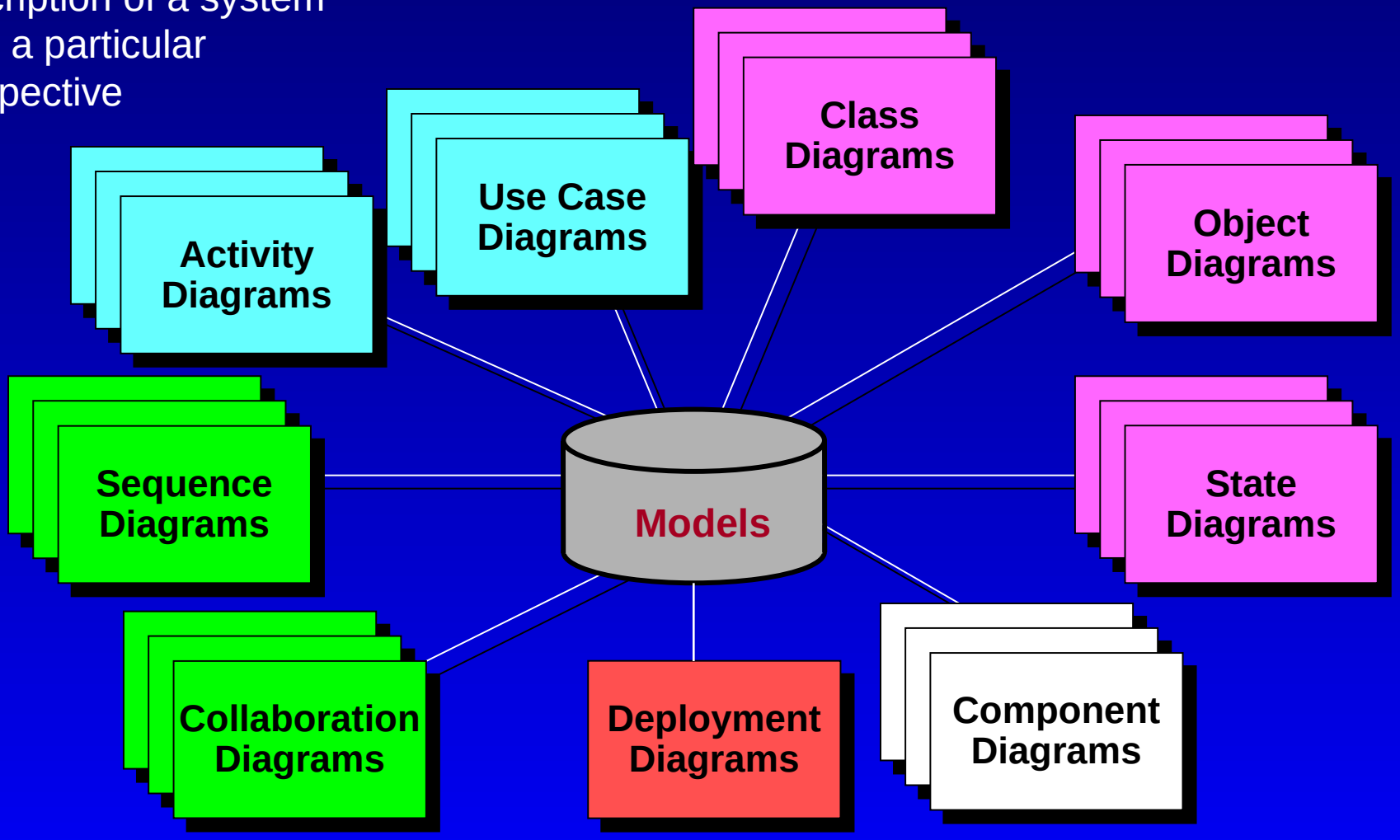
Visual modeling improves our ability to manage software complexity

Copyright © 1998 Rational Software, all rights reserved

R

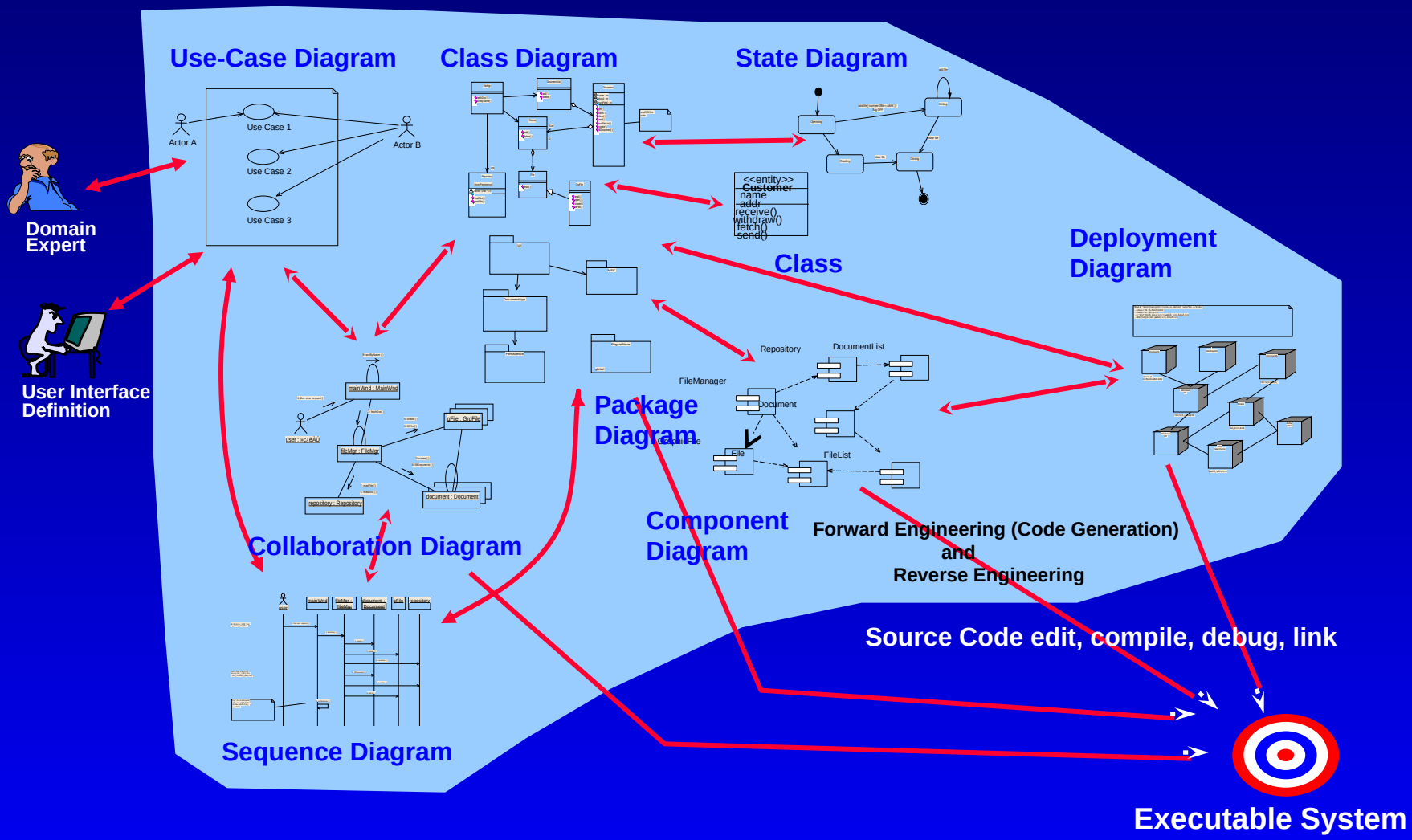
Diagrams Are Views of a Model

A *model* is a complete description of a system from a particular perspective



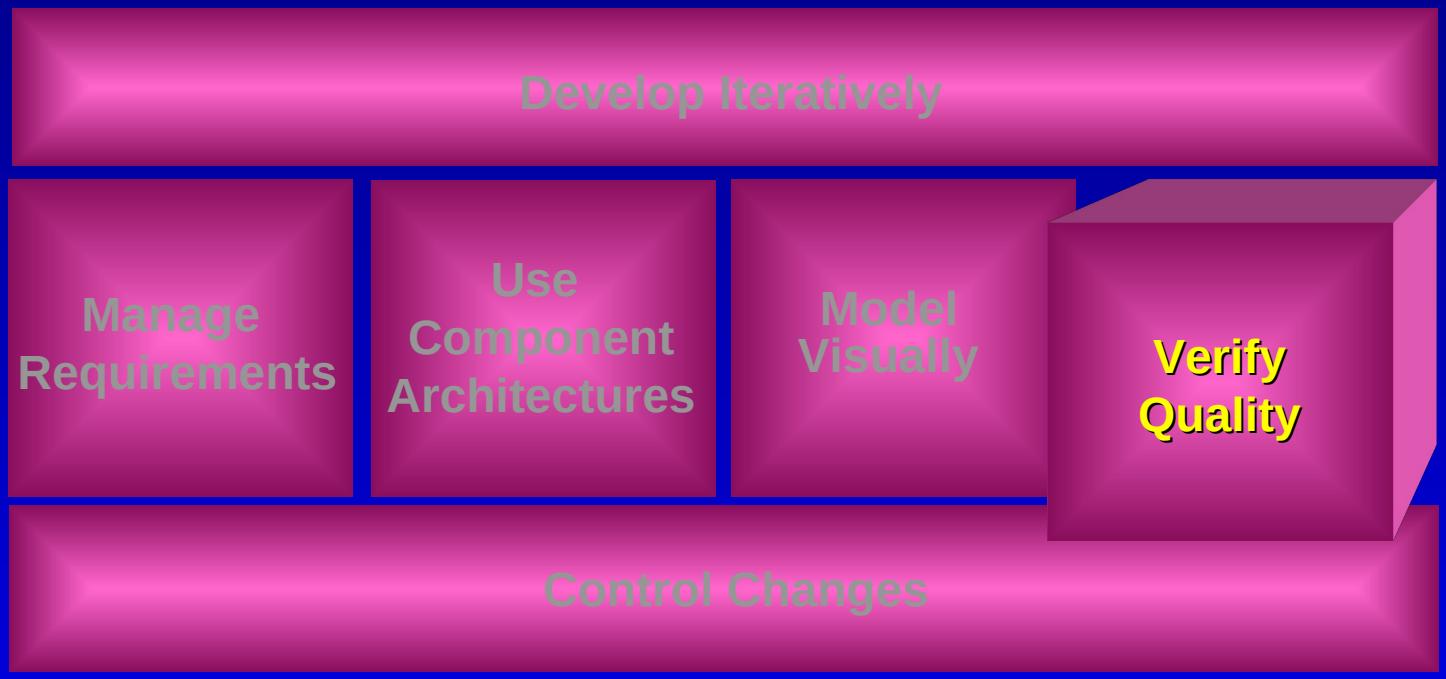
Copyright © 1998 Rational Software, all rights reserved

Visual Modeling Using UML Diagrams



Copyright © 1998 Rational Software, all rights reserved

Practice 5: Verify Software Quality

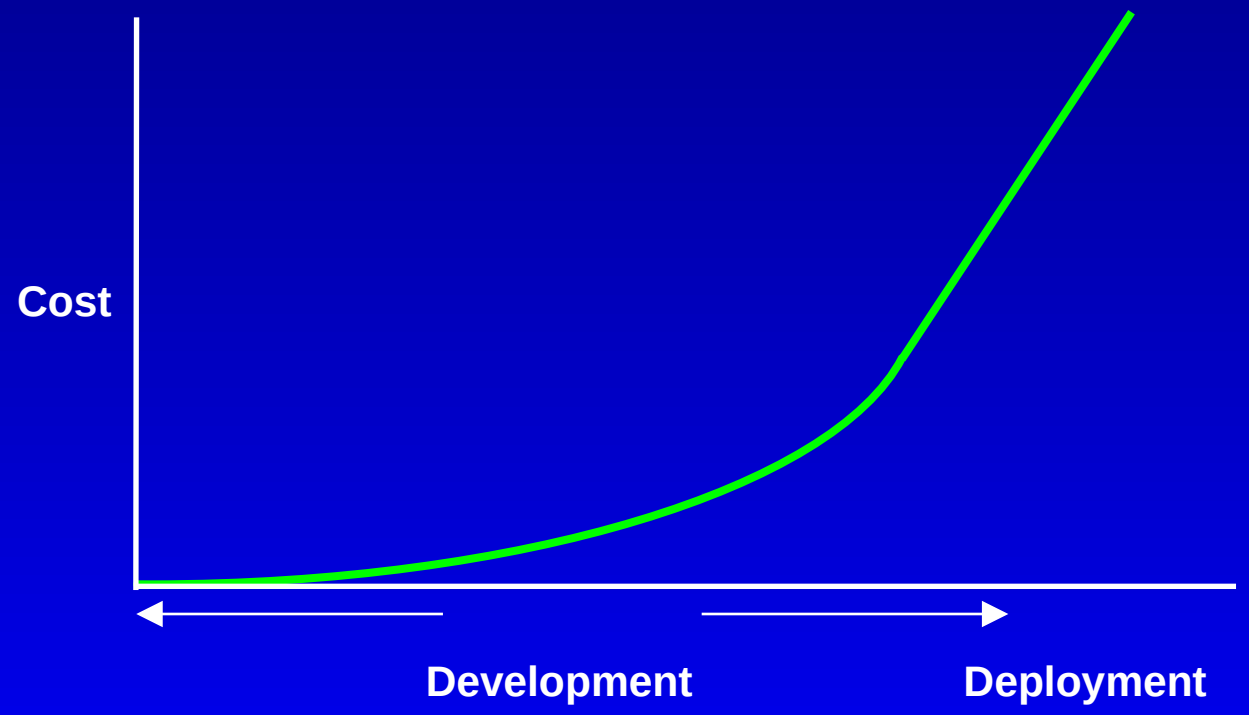


Copyright © 1998 Rational Software, all rights reserved

R

Practice 5: Verify Software Quality

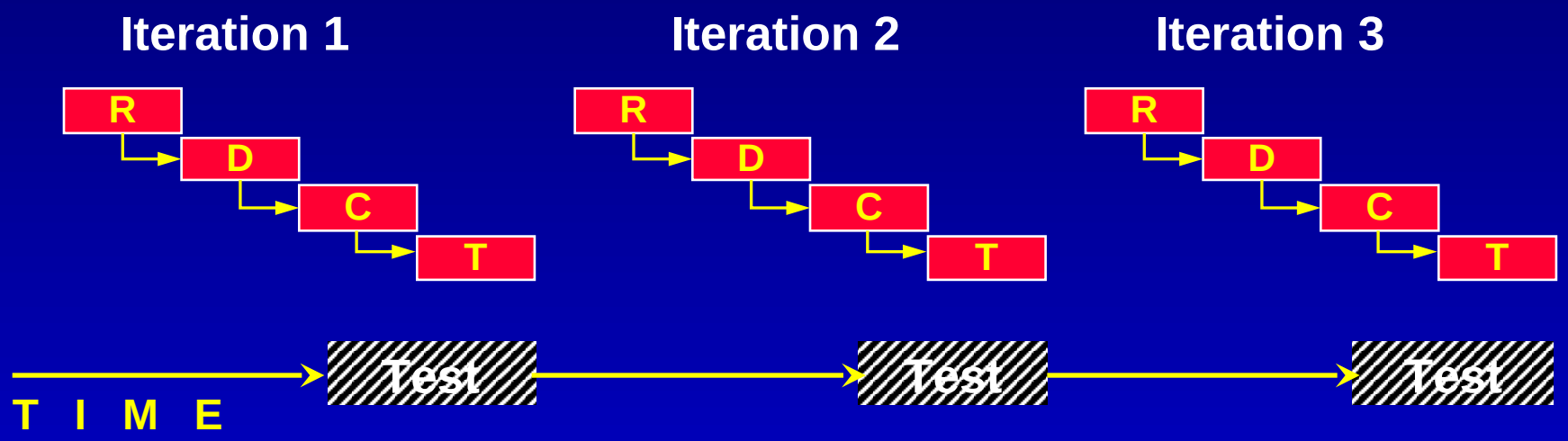
Software problems are 100 to 1000 times more costly to find and repair after deployment



Copyright © 1998 Rational Software, all rights reserved

R

Iterative Development Permits Continuous Testing



Test Life Cycle

Plan
Design
Implement
Execute
Evaluate

Plan
Design
Implement
Execute
Evaluate

Plan
Design
Implement
Execute
Evaluate

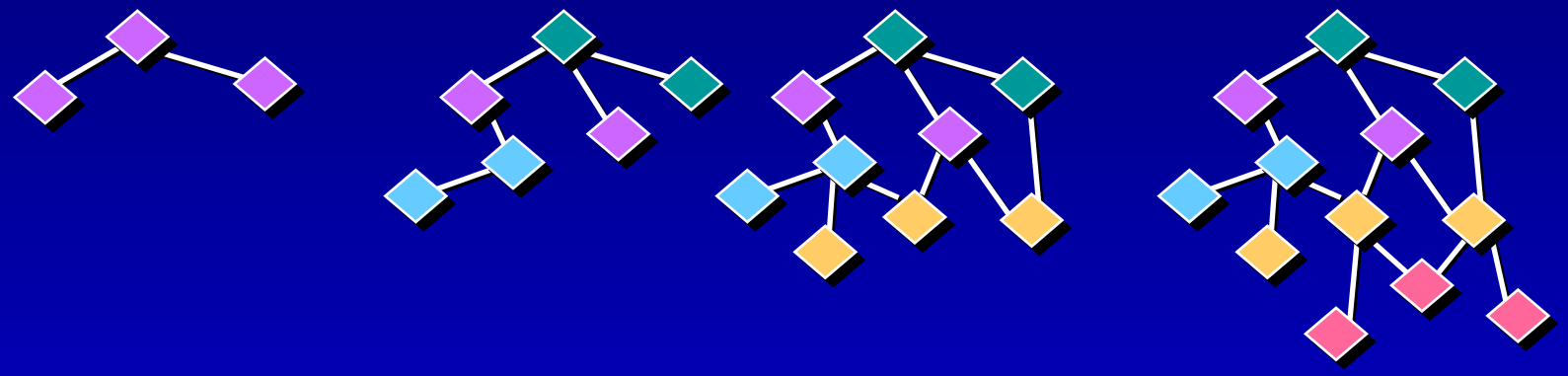
Copyright © 1998 Rational Software, all rights reserved

R

Testing in an Iterative Environment

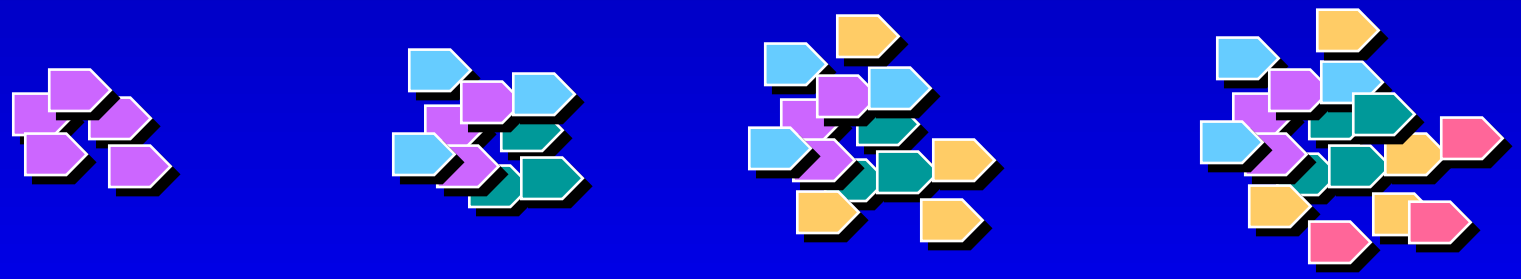
Requirements

Iteration 1 Iteration 2 Iteration 3 Iteration 4



Automated Tests

Test Suite 1 Test Suite 2 Test Suite 3 Test Suite 4



Copyright © 1998 Rational Software, all rights reserved

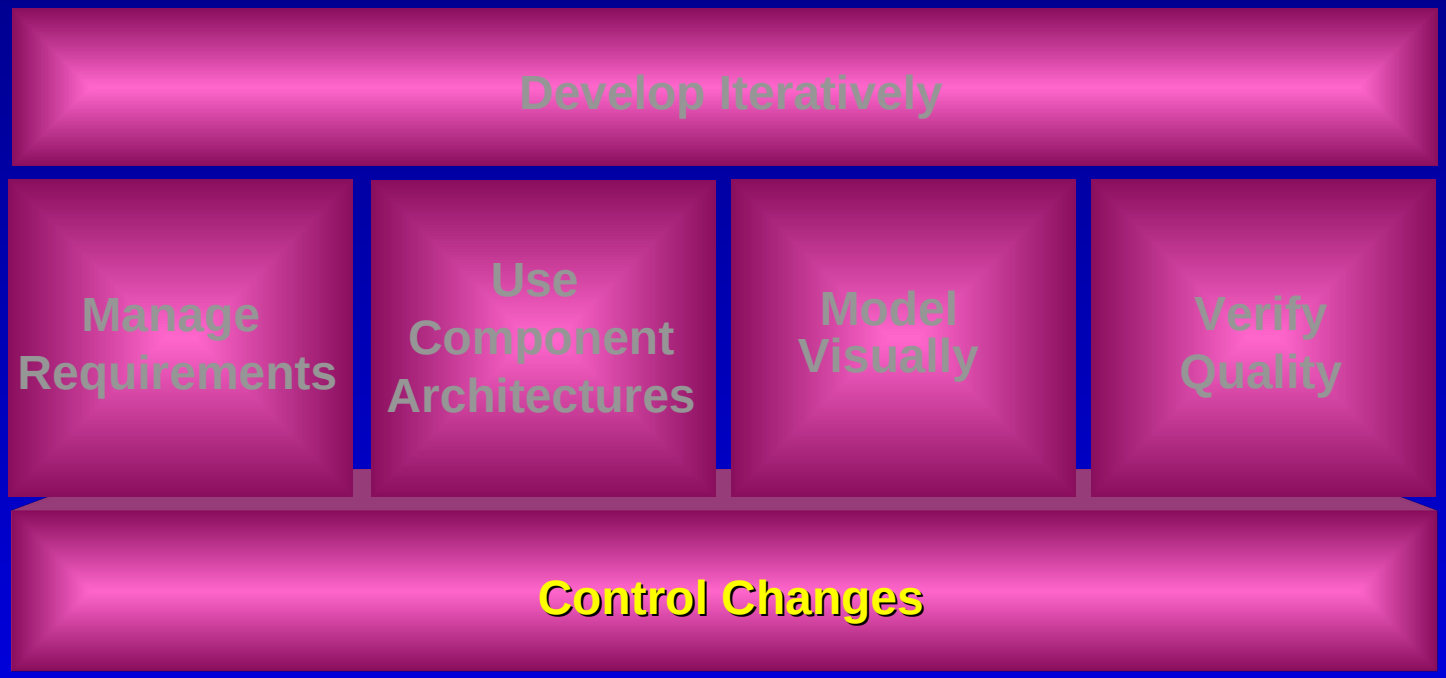
Dimensions of Software Quality

Type	Why?	How?
Functionality	Does my app do what's required?	Test cases for each scenario implemented
Reliability	Does my app leak memory?	Analysis tools and code instrumentation
Application Performance	Does my app respond acceptably?	Check performance for each use case/scenario implemented
System Performance	Does my system perform under production load?	Test performance of all use cases under authentic and worst-case load

Copyright © 1998 Rational Software, all rights reserved

R

Practice 6: Control Changes to Software



Copyright © 1998 Rational Software, all rights reserved

R

Practice 6: Control Changes to Software

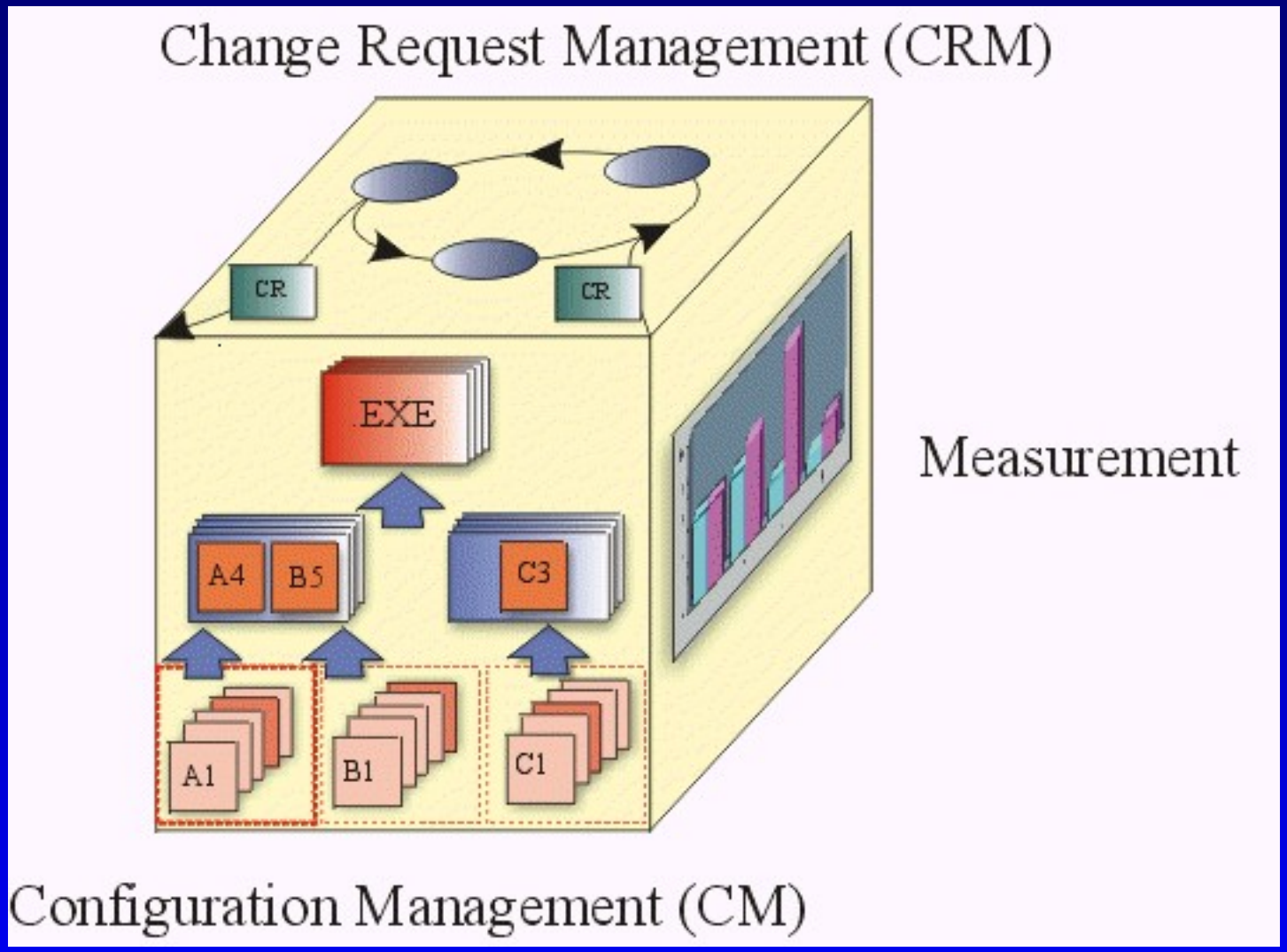
- Multiple developers
- Multiple teams
- Multiple sites
- Multiple iterations
- Multiple releases
- Multiple projects
- Multiple platforms

Without explicit control,
parallel development degrades to chaos

R

Copyright © 1998 Rational Software, all rights reserved

Three Major Aspects of a CM System



Copyright © 1998 Rational Software, all rights reserved

R

Concepts of Configuration & Change Management

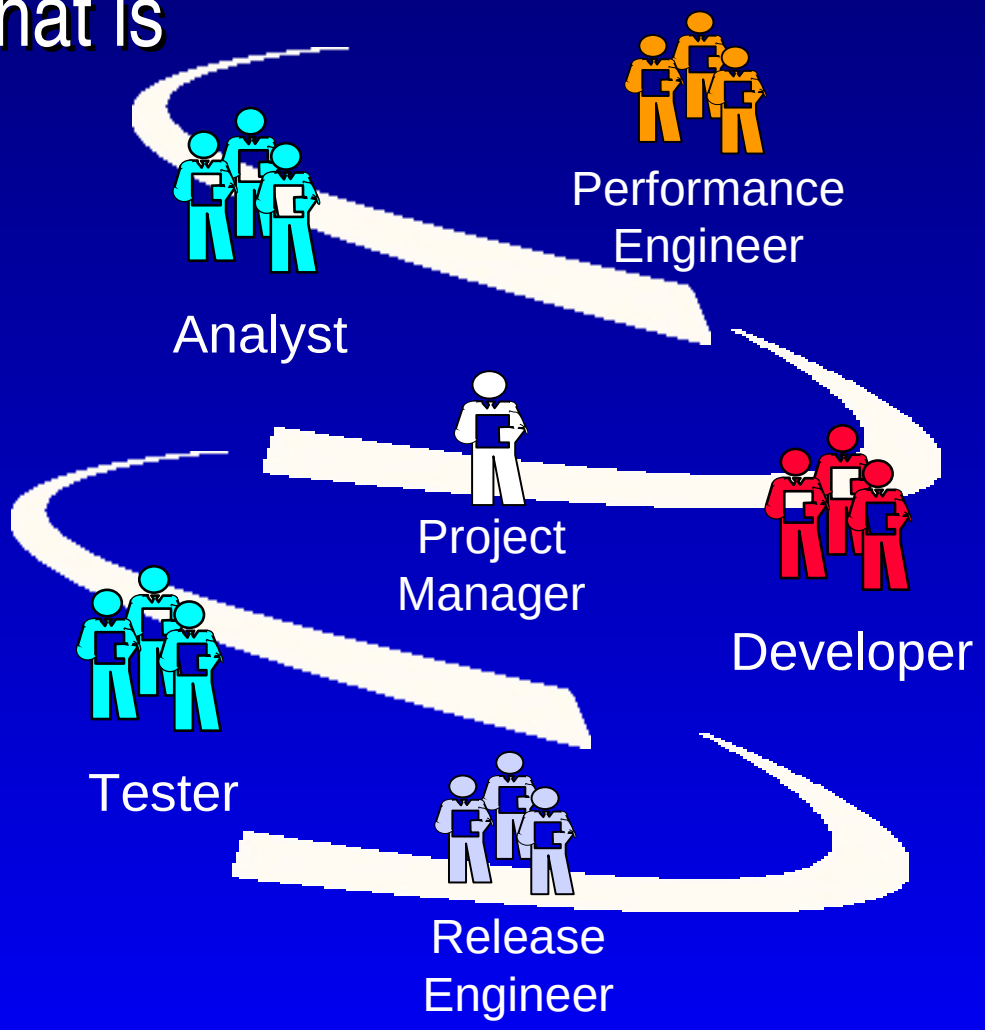
- Decompose the architecture into subsystems and assign responsibility for each subsystem to a team
- Establish secure workspaces for each developer
  - Provide isolation from changes made in other workspaces
  - Control all software artifacts - models, code, docs, etc.
- Establish an integration workspace
- Establish an enforceable change control mechanism
- Know which changes appear in which releases
- Release a tested baseline at the completion of each iteration

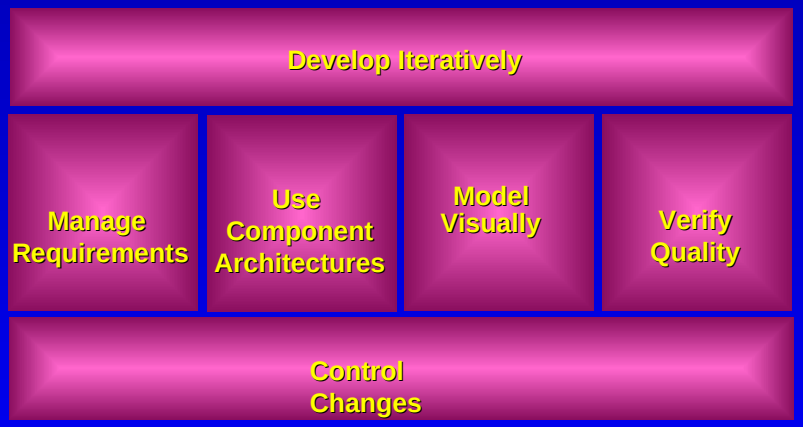
Copyright © 1998 Rational Software, all rights reserved

R

Summary: Best Practices of Software Engineering

- The result is software that is
  - On Time
  - On Budget
  - Meets Users Needs





Copyright © 1998 Rational Software, all rights reserved